Music and Audio Programming Final Project: The Crystalliser

George McVicar - Student ID: 110074605

April 2026

1 Introduction

'Freeze' functions in audio effects involve a process of sampling, which allows the user to modulate the recorded ('frozen') audio playback to musical effect. Such a freeze function often appears in reverb plugins and pedals, but is rarer for 'clean' playback, though it can produce some interesting musical textures.

The plugin presented in this report, titled 'The Crystalliser', is based on the freeze function of the Particle pedal by Red Panda[4] as an inspiration, but with added modulation parameters and a reverb class also installed. In addition to the freeze buffer parameter, The Crystalliser is also capable of varying loop length, LFO modulation of this length, 'Persistence' (the amount of 'hold' on the looped audio), dry/wet blending, and reverb parameters. The Crystalliser is novel in its combination of freeze functions with an LFO parameter and applied 'freeze-only' reverb.

2 Background

One example of a clean freeze effect can be found in the Particle delay pedal from Red Panda[4]. This guitar pedal is a granular delay effects unit, using granular synthesis in real time to manipulate audio input. A feature of this pedal is the 'freeze' function which allows the user to capture some of the sound input and loop it continuously, with an amplitude threshold. For example, a user can capture audio at a low threshold, loop it, and bring the threshold up to enable a drone-like backing which does not capture any new audio going into the freeze function.

Freeze effects are an application of 'circular buffer' design, which is also used in delays, reverbs, and granular synthesis. A buffer is a contiguous block of memory, whereby read and write pointers point to specific locations within a block. The difference in read and write pointer location determines the amount of delay between audio input and audio output. Such a model is therefore equivalent to difference equation representations, because the circular buffer implements a delay of z^{-n}[5].

However, where freeze effects differ from delay effects is through the use of the 'state machine'. Most audio effects are 'stateless' in the sense that they process each sample in the same way. By contrast, freeze effects are state*ful* in that they are either in a recording state or in a playback state. The presence of the state machine can create challenges (such as unwanted 'clicking' sounds between states), but also opportunities (such as the potential to create drone-like backing effects, whereby the frozen audio is not affected by any new audio input).

'Freeze' effects are also sometimes seen within reverb effects units, such as Ableton's standard reverb plugin[1]. In these reverb effects units, the freeze function generates a reverberated loop from the audio input signal, creating a continuous sound. The resulting effect is similar to an atmospheric hum, which can be used to add space to a mix (for example, so that there is never complete silence in a piece of music). However, clean playback of frozen effects is much rarer among audio effects processing, despite the fact that it has the potential to create some very interesting musical effects.

This plugin also makes use of the reverb class in JUCE. The class performs a reverb effect on a stream of audio data, in this case as a simple stereo reverb. The reverb model used in this class is similar to that suggested by J.A. Moorer[2], which combines an FIR delay line (for simulating early reflections) with six parallel comb filters with lowpass filtering and an all-pass filter (to increase the reflection density) and a delay filter to simulate late reverberation. Moorer's approach attempts to emulate the reverb heard in a six-surfaced room (including floor and ceiling), and is particularly effective at replicating the kind of reverb one would hear in a concert hall. In this plugin, the Moorer-style reverb is added exclusively to the frozen signal as a deliberate design choice.

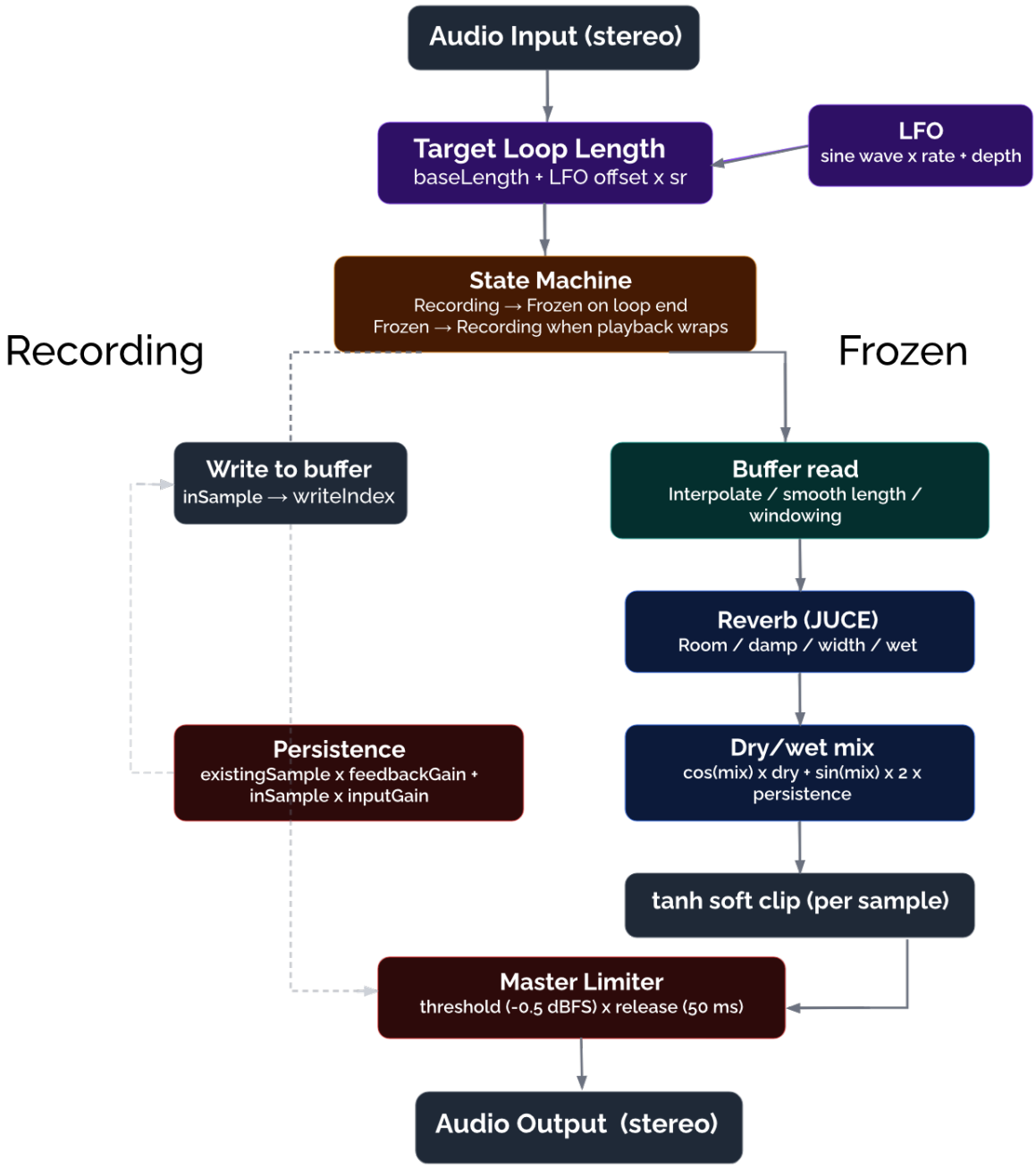

Figure 1: Signal Flow Diagram of The Crystalliser

3 Signal Flow

Figure 1 shows the signal flow from audio input to audio output. Put into words, audio input is sent to a loop, whose length is modulated by an LFO. The state machine then records live audio into a circular buffer, 'freezes' it, and plays it back when the loop completes. For the frozen audio, each sample is read back with interpolation and a cosine fade window to avoid clicks, passed through reverb, then blended with the live input using an equal-power dry/wet mix scaled by the Persistence parameter. A soft clipper (`tanh`) is also added to the frozen signal which adds a small amount of harmonic distortion to large amplitudes. Persistence also feeds audio back into the buffer itself, controlling how long the frozen texture holds before dissolving. Everything exits through a master limiter before being sent to the audio output.

4 Methodology

The plugin was created in JUCE using three core components:

- `juce::AudioProcessor` — the DSP engine
- `juce::AudioProcessorEditor` — the GUI
- `juce::AudioProcessorValueTreeState` — parameter management

To create a freeze effect, a rolling buffer stores the incoming audio. The buffer is initialised with two channels and a capacity of five seconds:

```
CrystalliserBuffer.setSize(2, (int)(sr * 5.0));
```

Listing 1: Rolling buffer initialisation

Incoming audio is continuously written to this buffer. When the state machine is in its frozen state, audio is instead read back using a playback index, and looping is achieved with a wrapping index that resets once it reaches the current loop length:

```
CrystalliserBuffer.setSample(ch, writeIndex, inSample);
...
playbackIndex += 1.0f;
if (playbackIndex >= currentLoopLength) { playbackIndex = 0; state = Recording; }
writeIndex = (writeIndex + 1) % CrystalliserBuffer.getNumSamples();
```

Listing 2: Write path and looping playback index

The loop length is modulated by a sine wave LFO, with parameters exposed for both rate and depth:

```
float lfoValue = std::sin(lfoPhase);
```

Listing 3: LFO modulation of loop length

This gives the looping effect a 'breathing' quality rather than a static set of repetitions.

Experimenting with the feedback coefficient in the buffer write-back produced interesting timbral effects, essentially controlling how many repetitions a loop would play before decaying. A parameter called 'Persistence' was therefore created to expose this value to the user. At 0% (`feedbackGain = 0.8f`, `inputGain = 0.2f`) new audio energy dominates, causing the loop to refresh rapidly; at 100% (`feedbackGain = 0.995f`, `inputGain = 0.005f`) the frozen buffer is held almost indefinitely:

```
CrystalliserBuffer.setSample (ch, (int)readPos % bufferSize,
                    (existing * feedbackGain) + (inSample * inputGain));
```

Listing 4: Persistence feedback coefficients

Finally, reverb was implemented using the `juce::Reverb` class and applied only to the frozen signal:

```
float revL = frozen, revR = frozen;
reverb.processStereo(&revL, &revR, 1);
```

Listing 5: Reverb applied to the frozen signal only

The reverb parameters include `roomSize`, where 0.0 is small, 1.0 is big; `damping`, where 0 is not damped, 1.0 is fully damped; `width`, where 0.0 is narrow and 1.0 is very wide; and `wetLevel`, 0.0 to 1.0, where 0.0 has no wet signal, and 1.0 is fully wet.

Finally, an overall Dry/Wet mix parameter was implemented using a constant-power crossfade, where the dry signal is true bypass. By using complementary sine and cosine functions, the combined energy remains constant across the full range of the knob, avoiding the volume dip that a simple linear blend would produce at the midpoint:

```
float dryG = std::cos(mix * juce::MathConstants<float>::halfPi);
float wetG = std::sin(mix * juce::MathConstants<float>::halfPi);
```

Listing 6: Constant-power dry/wet crossfade

5 Issues and Optimisation

A significant challenge in creating this plugin was reducing the 'clicks' between samples. The clicks occurred due to discontinuities when the playback index wrapped from the end of the buffer back to the start. Note that the LFO and parameter smoothing make `currentLoopLength` a floating-point number, meaning the buffer is read between samples; linear interpolation was therefore used to calculate the fractional amplitude between `indexA` and `indexB`. To resolve the clicking, a dynamic crossfade was implemented, creating amplitude windowing relative to the current loop length (10%), capped at 100 ms. The linear fade was also replaced by a cosine S-curve to make the volume transition sound more natural:

```
// Dynamic fade size (capped at 100ms)
float fadeSize = juce::jmin (currentLoopLength * 0.1f, 4410.0f);

// Linear interpolation
int   indexA = (int)readPos % bufferSize;
int   indexB = (indexA + 1) % bufferSize;
float frac   = readPos - (float)((int)readPos);
float frozen = CrystalliserBuffer.getSample (ch, indexA)
             + frac * (CrystalliserBuffer.getSample (ch, indexB)
                 - CrystalliserBuffer.getSample (ch, indexA));

// Dynamic Windowing
float windowGain = 1.0f;
if      (readPos < fadeSize)
    windowGain = readPos / fadeSize;
else if (readPos > (currentLoopLength - fadeSize))
    windowGain = (currentLoopLength - readPos) / fadeSize;

// Cosine S-Curve
windowGain = juce::jlimit (0.0f, 1.0f, windowGain);
frozen *= (0.5f - 0.5f * std::cos(juce::MathConstants<float>::pi * windowGain));
```

Listing 7: Linear interpolation, dynamic windowing, and cosine smoothing

Clicking also appeared when altering the loop length parameter, because the buffer length was jumping instantly to new values mid-cycle. This was resolved by implementing a single-pole IIR low-pass filter applied to the parameter value. Rather than snapping to a new value, `currentLoopLength` changes smoothly, further reducing the clicks:

```
targetLoopLength = (int)(juce::jlimit(0.05f, 5.0f,
    baseLength + (lfoValue * baseLength * lfoDepth * 0.5f)) * sr);
...
//Single-pole IIR smoother
currentLoopLength = 0.8f * currentLoopLength + 0.2f * (float)targetLoopLength;
```

Listing 8: Single-pole IIR smoother for loop length

There were also issues of gain staging within the plugin. At high Persistence settings the wet looped signal was very quiet, but at low settings the signal came out very loud. This occurred because at low persistence, new audio is continuously routed into the frozen buffer, causing accumulative energy to stack up and spike in volume. An inverse persistence gain parameter was therefore created: when Persistence is low (0.0) `persistenceGain` is taken down to 0.2, which equates to a reduction of approximately 14 dB, and when Persistence is high (1.0) the gain is left at 1.0. A fixed gain of 8.0 (approximately 18.06 dB) is also applied to the wet signal to compensate for the overall low amplitude of the frozen output. This boost is then kept in check by the `tanh` soft clipper, which adds a small amount of harmonic distortion at high amplitudes rather than hard clipping:

```
float persistenceGain = 0.2f + (persistence * 0.8f);
...
buffer.setSample (ch, sample,
    std::tanh ((dryG * inSample) + (wetG * (frozen * 8.0f * persistenceGain))));
```

Listing 9: Inverse persistence gain applied to output

Finally, a `juce::dsp::Limiter` was added at the end of the plugin chain to level the wet signal. Placing the limiter inside the sample loop initially caused total silence due to overprocessing; it was therefore moved to block level, outside the loop:

```
masterLimiter.prepare(spec);
masterLimiter.setRelease(50.0f);     // Smooth 50ms release
masterLimiter.setThreshold(-0.5f);   // Hard ceiling just under 0 dBFS
```

Listing 10: Master limiter at block level

6 Analysis

Spectrograms of the audio output can be seen in Figures 2, 3, and 4. For the purposes of demonstration, a square wave C5 (523.25 Hz) was used as the baseline to best capture the effect on the harmonics of the sound. Note that these harmonics are visible as discrete horizontal lines. In Case A (Figure 2), all parameters are set to 0.0 to show the true bypass signal:

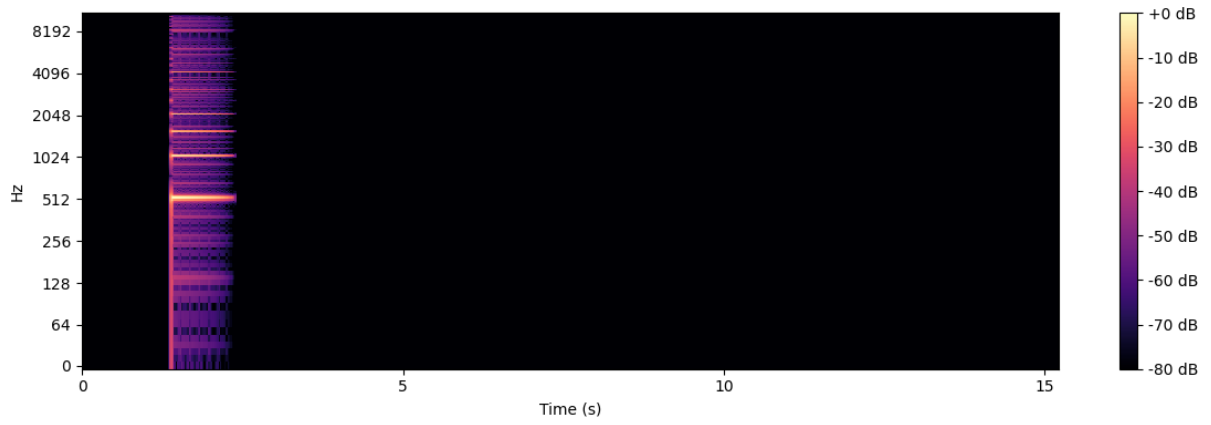

Figure 2: Case A (true bypass) spectrogram

Case B (Figure 3) has Mix set to 0.8, Persistence to 0.8, Length to 0.24, Mod Rate to 0.1, and Mod Depth to 1.0. All reverb parameters are set to 0.0. Comparing to Case A, the same harmonics are clearly repeated, but the LFO gives a 'warping' quality that morphs over time. By around 10 seconds, the original frozen material has almost faded, but the strength of the signal begins to return between 10–15 seconds. This is because the new degraded material is now being frozen, with a different spectral character. The repetitions between 10–15 seconds also appear to have some attack, which is an artefact of the dynamic crossfading implemented to reduce unwanted clicking sounds. This warping quality is one of the more unusual aspects of this plugin.

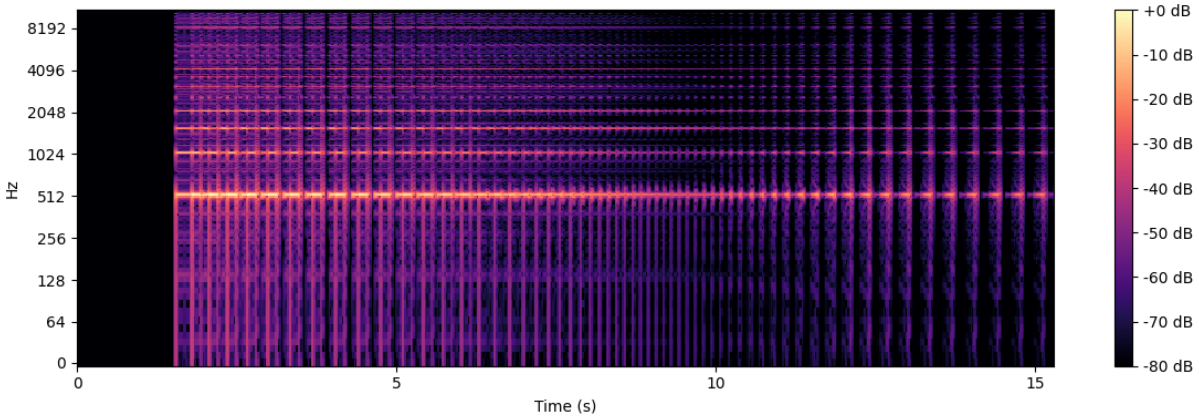

Figure 3: Case B (LFO added) spectrogram

Case C (Figures 4 and 5) showcases the extremities of the plugin over a full minute-long recording, with parameters altered to maximise the complexity of the output. With Persistence set to 1.0, and the reverb fully activated, the everlasting character of the loop is clearly visible, with the harmonic content sustained throughout the entire recording. Also note how the reverb smears energy across time and adds diffuse low-level content between the harmonic peaks.

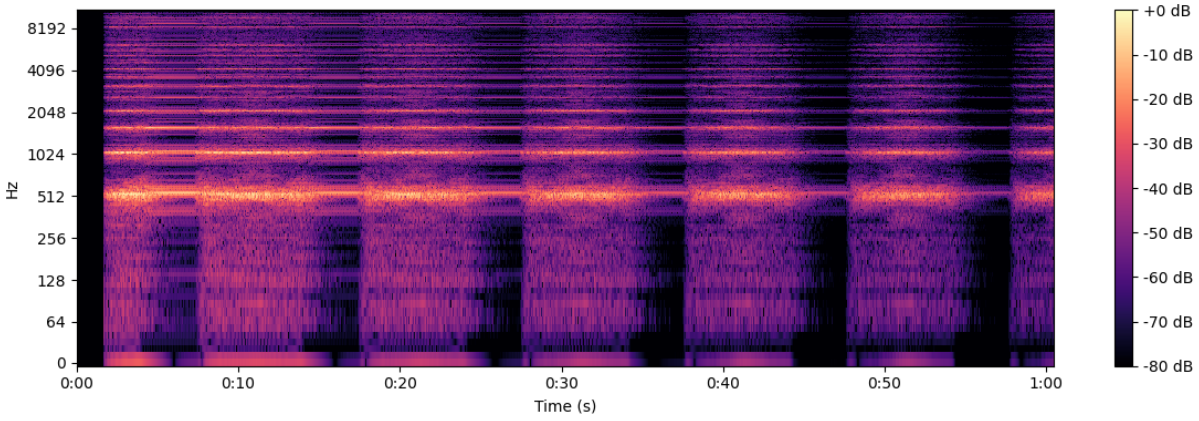

Figure 4: Case C (maximised persistence and reverb) spectrogram

The waveform visualisation in Figure 5 more clearly shows how the amplitude envelope of the output varies over time. This is an unexpected and indirect effect of the LFO. Because the cosine fade window is sized relative to the current loop length (10%), the LFO's modulation of loop length causes the windowing envelope to change in proportion, producing a periodic variation in amplitude when the loop length is a small value.

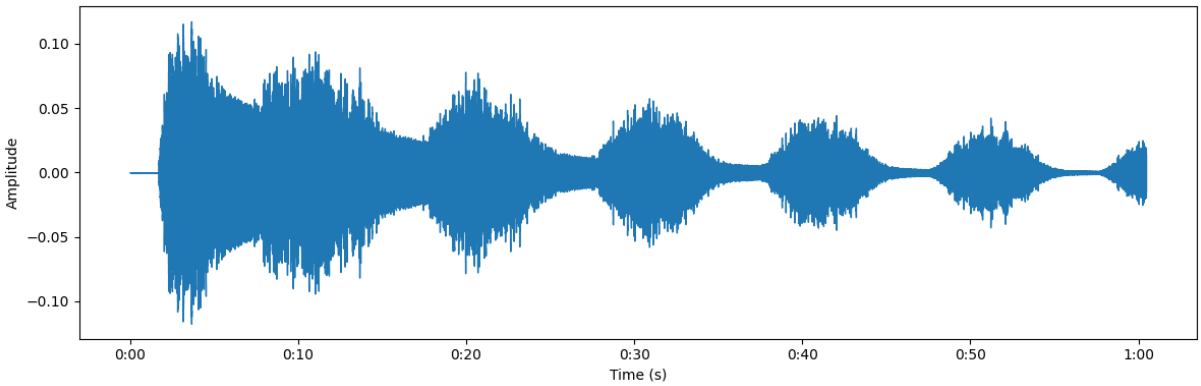

Figure 5: Case C (maximised persistence and reverb) waveform

A summary of the parameters used for Cases A, B, and C is seen in Table 1:

Parameter	**Case A**	**Case B**	**Case C**
Duration (s)	15	15	60
Length	0.00	0.24	0.07
Mix	0.00	0.80	1.00
Persistence	0.00	0.80	1.00
Mod Rate	0.00	0.10	0.10
Mod Depth	0.00	1.00	1.00
Room	0.00	0.00	1.00
Damp	0.00	0.00	0.00
Width	0.00	0.00	1.00
Rev Wet	0.00	0.00	1.00

Table 1: Parameter settings for Cases A, B, and C.

7 Results

This plugin creates some unusual textures with a lot of parameter control for the user. Such a plugin could therefore be used to great effect in ambient or experiential music, or in sound design. For example, to represent gemstones or glacial landscapes in visual media. Using the LFO parameters to modify the loop length creates some highly novel sounds that are rarely heard in plugins. Moreover, the 'persistence' parameter is also very versatile — at low persistence, the plugin works almost like a delay, but at high persistence the sonic character morphs into very unpredictable patterns that can inspire new musical ideas.

However, due to its complexity this plugin works best only with certain audio inputs. Any input that is too spectrally rich, such as with multiple instruments or anything with a high number of harmonics, produces a dense, incoherent output that is difficult to use musically. In comparison to the Particle by Red Panda[4], which served as the inspiration for this project, the sound quality of the plugin is more unusual, and more suited to producing soundscape-like textures, especially due to the LFO modulation and reverb class. However, the Particle has some additional parameters (such as pitch manipulation) which gives the user greater control over the audio output.

The overall sound quality of this plugin is 'crystalline', capable of producing sharp, glacial textures and sparkling shimmers. The looping repetitions also add an everlasting or enduring character, especially at high persistence levels. The name 'The Crystalliser' was therefore chosen to capture both the plugin's glistening effects and this sense of permanence. To give the plugin a sleek-looking GUI with the 'crystalline' theme in mind, diamond-shaped sliders were created with bright blue outlines and a cold colour palette backdrop. The `LookAndFeel` class was restructured with assistance from ChatGPT[3] to neaten the placement of sliders. The final GUI can be seen in Figure 6:

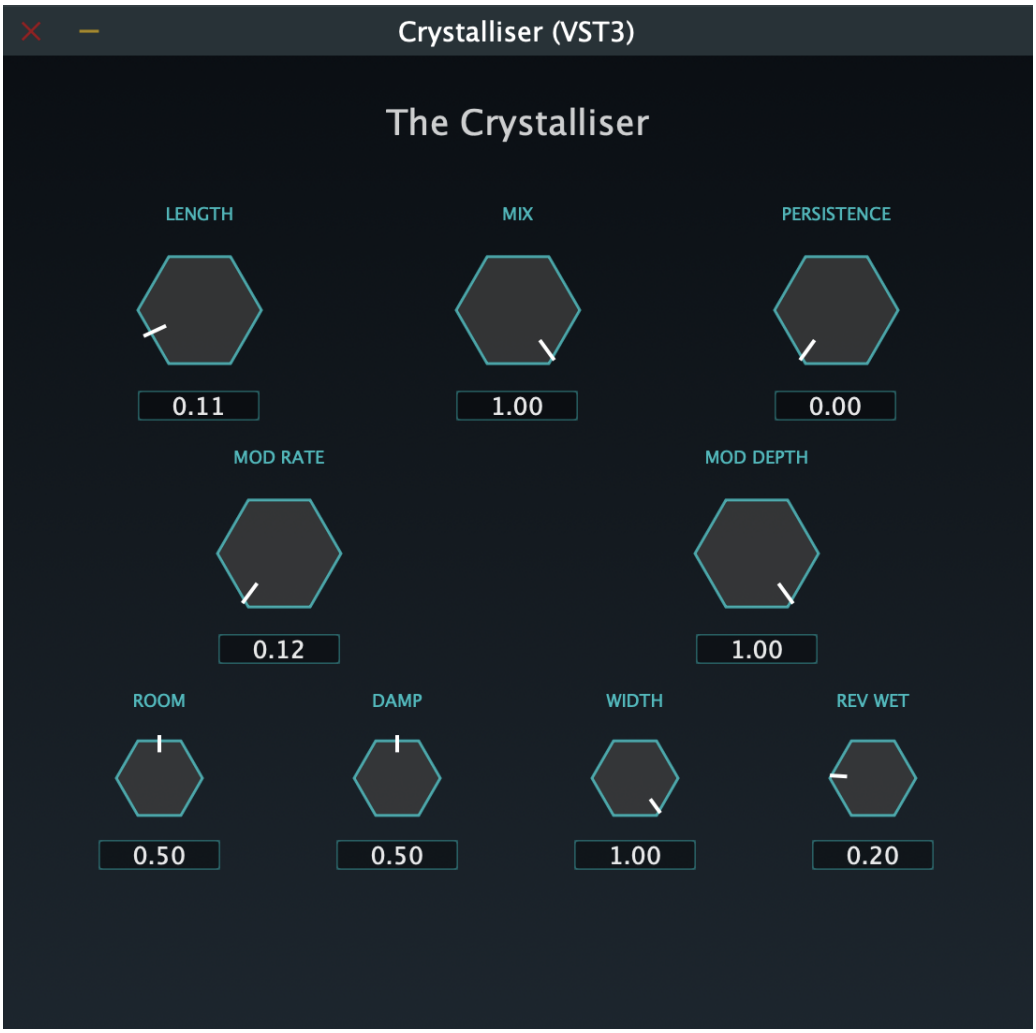

Figure 6: GUI for The Crystalliser.

8 Conclusion

This report describes the creation of a novel 'freeze' plugin within JUCE. The LFO modulation of the freeze loop length, the Persistence parameter, and the integrated reverb class give the user a great degree of creative control over the audio output, producing some very unusual textural effects.

In future work, integrating EQ parameters would be a worthwhile addition. For example, the ability to filter the frozen output (such that only frequencies above a high-pass cutoff are frozen) would give the user even greater control. A single-pulse 'kill switch' to end repetitions immediately would also improve user experience, as the user currently has to manually reduce Persistence to 0.0 to dissolve the current loop. Finally, it would be worthwhile to explore a 'reverse' playback mode for the frozen audio material, for creating even more unusual frozen textures.

References

[1] Ableton. Live audio effect reference. `https://www.ableton.com/en/manual/live-audio-effect-reference/`, 2026. Accessed: April 2026.

[2] J. A. Moorer. About this reverberation business. *Computer Music Journal*, 1979.

[3] OpenAI. ChatGPT. `https://chat.openai.com`, 2026. Accessed: April 2026.

[4] Red Panda Lab. Particle. `https://www.redpandalab.com/products/particle`, 2025. Accessed: April 2026.

[5] Joshua D. Reiss and Andrew McPherson. *Audio Effects: Theory, Implementation and Application*. CRC Press, 2014.